

THE BOSS BATTLE REVIEW GAME

Twelve tasks. At the end you have a real game — health bars, sprites, sound, screen shake — where **your** review questions are the weapon. One file. No student logins, no subscription, and it runs with the wifi down.

12

tasks, each ending in something
you can play

1

finished game included

0

accounts anyone has to make

This is the part of AI nobody shows teachers.

Not a faster worksheet. Not a lesson plan in thirty seconds. An actual piece of software, built by you, in an evening, about the exact thing you taught this week — and it belongs to you afterwards.

No coding background. About ninety minutes.

Nothing installed, nothing to sign up for, no build step. One file that opens by double-clicking it. Swap in your own questions by editing eight lines at the top.

Inside

Twelve tasks with the exact prompts written out in full · the finished game, playable today · the six failures you will hit and the sentence that fixes each · five ways to run it with a class · how to write questions worth fighting over · and a straight page on building in a famous game's genre without stealing from it.

BEFORE YOU BEGIN

The twelve tasks at a glance

Every task ends with something on screen. If a task does not, stop there and fix it — the next one builds on it.

#	Task	Ends with
1	Pick the fight	A boss, a unit, eight questions
2	The stage	A window with a floor and a sky
3	The two fighters	Pixel art you drew in text
4	Health bars that drain	Two bars that slide, not jump
5	The question and four answers	Something to click
6	A right answer lands a hit	The boss losing health
7	A wrong answer hits back	Stakes
8	Make it feel good	Shake, flash, damage numbers
9	Sound, with no sound files	A hit you can hear
10	Winning and losing	Two endings and a rematch
11	The class record	A time that survives the tab closing
12	Everyone can play it	Keyboard, touch, Chromebook

Stop anywhere after task 7

Tasks 1 to 7 give you a working game. Tasks 8 and 9 are what make a class shout at the screen, and they are the ones worth staying up for.

HONESTLY

You already have Blooket. Why build one?

Fair question, asked first, because a teacher who has used Gimkit will ask it in the first ten seconds.

	The ones you know	The one you build
Student logins	Codes, accounts, or both	None. It is one file on your screen.
Needs the internet	Yes	No. Works with the wifi down.
Cost	Free tier, then a yearly fee	Nothing, forever
Blocked by your district?	Sometimes	It is a local file, so no
Student data leaves the room	Yes, to their servers	Never. There is nothing to send.
The game itself	Their template, your questions	Yours. Change any rule you like.
Setup time	Minutes	An evening, once

Be honest about the trade. Their tools are faster to start and have polish you will not match on a Tuesday. What you get instead is a game that needs nothing from anyone, costs nothing ever, cannot be taken away when a subscription lapses or a district blocks a domain, and can be changed on the morning you decide the rules are wrong.

The part that surprises people

Classes care more about a game their teacher made than a polished one from a company. Say you built it. They will ask how, and that is a better conversation than any lesson on digital citizenship.

A game is a machine that remembers what is happening

This is what separates a game from a worksheet with pictures, and it is the thing worth actually learning here.

A worksheet has no state

Questions sit there. Answering one changes nothing about the others. A printable quiz and a fancy digital quiz are the same machine.

A game has state, and everything reads from it

One object holds the whole situation: how much health each side has, which question is up, how many right in a row, whether the screen is shaking. **Every frame, the game draws that object. Every answer changes it.**

That is the entire architecture, and once you have seen it you can build any game.

Three parts, and you will build them in this order

The state — one object. Health, question number, streak.

The loop — sixty times a second, nudge the animations along and draw everything.

The events — a click or a key changes the state. The loop notices on the next frame.

Why to say this to the AI out loud. Ask for "a quiz game" and you often get one giant function that redraws the page on every click, and it becomes unchangeable by task 6. **Ask for a state object, a draw function and a game loop** and you get something you can still be editing in March.

WHAT YOU NEED

Before task 1

An AI chat you are signed in to

Any of them. You paste prompts and read what comes back. You are at the keyboard the whole time.

A browser, and a folder

The game is one HTML file you save wherever you keep things. Double-click it and it runs. That is the whole deployment.

Eight questions you already have

Off a quiz, a review sheet, an exit ticket. Four answer choices each. Do not write new ones tonight — use something you already trust, so the first version is about the game rather than about the content.

What you do not need

Node, npm, a terminal, an account, a server, a credit card, an art program, or any music. The sound in this game is generated by the browser, which is why there are no audio files to lose.

If you only have twenty minutes tonight, do tasks 1 to 4. You will have a boss, a hero and two health bars on screen, which is the moment this stops feeling theoretical.

TASK 1

Pick the fight

Four minutes with a pen. The only task the AI cannot do for you.

Three decisions

What are they fighting? Name the boss after the thing they find hard. The Water Cycle. Fraction Island. The Comma Splice. A unit that beats students becomes a monster that beats students, and that joke does a lot of work.

Which eight questions? Pull them from something you already use.

How long should a fight take? Twelve correct answers is about ten minutes of class with discussion. That is the default and it is a good one.

Write the questions in this shape now

```
{ q: 'Water turning from liquid into vapor is called what?',  
  a: ['Evaporation', 'Condensation', 'Precipitation', 'Collection'],  
  ok: 0 }
```

ok counts from zero, so 0 is the first answer. Getting this shape right now means task 5 is a paste rather than a negotiation.

The mistake that makes a boring game

Three obviously wrong answers and one obviously right one. A class shouts the answer in two seconds and the game is over in ninety. There is a whole page on this later; for now, make the wrong answers the mistakes your students actually make.

Task 1 ends when

You have a boss name and eight questions written in that shape.

TASK 2

The stage

A window to fight in, and proof the loop is running.

PROMPT 1 · THE STAGE

Build a single HTML file, no frameworks and no build step, that runs by opening it in a browser. Put a canvas in it, 640 by 360, scaled to fit the page, with **imageSmoothingEnabled set to false** so pixel art stays sharp. Set up a **state object, a draw function, and a game loop using requestAnimationFrame** — I want those three things separate, not one big function. Draw a dark sky, a horizon band and a floor. Put a few stars in the sky at fixed positions so they do not flicker. Make something move every frame so I can see the loop is alive.

The three phrases doing the work

"no frameworks and no build step" — stops it reaching for React, which needs a terminal.

"a state object, a draw function, and a game loop" — the architecture from the last page, asked for by name.

"stars at fixed positions" — ask for random stars and they are redrawn in new places sixty times a second, which looks like static.

Task 2 ends when

You open the file and see a sky, a floor, and something moving.

TASK 3

The two fighters

Pixel art you can draw in a text editor, with no art program and no image files.

PROMPT 2 · SPRITES

I want pixel sprites **written as arrays of text**, one character per pixel, with a dot for transparent — so I can edit the art by typing. Write a function that takes one of those arrays, a color map, an x and y, and a pixel size, and draws it on the canvas. Then make two: a small hero standing bottom left, and a **big boss, at least fourteen pixels wide, on the right**. Give the boss a slow bob up and down so it looks alive. Put a soft colored glow behind it.

What a sprite looks like, so you can change it

```
hero: [  
  '...1111...',  
  '..111111..',  
  '.1122221.',  
  '.12211221.'  
]
```

Every 1 is one color, every 2 another, every dot is nothing. **Change a character, save, reload — the art changed.** That is the moment most teachers realise they can actually do this.

Make the boss big

Left alone the AI makes both fighters the same modest size and the screen looks like a chess problem. **A boss should be four or five times the hero.** Say so in the prompt, and say it again if it does not listen.

Task 3 ends when

Two characters are on the stage and the big one is breathing.

TASK 4

Health bars that drain

The single cheapest thing you can do to make software feel like a game.

PROMPT 3 · THE BARS

Add health to the state: the boss has 200, the class has 100. Draw a bar for each, with a label and the numbers beside it. Put the boss bar at the top under its name and the class bar at the bottom under the hero. **The bars must slide toward the real value a little each frame rather than jumping** — keep a separate "shown" number per bar and ease it toward the true one. Turn the bar yellow below half and red below a quarter.

Why the sliding matters more than it sounds

A bar that jumps from 200 to 186 is information. A bar that *drains* from 200 to 186 over half a second is a **hit landing**. It is four lines of code and it is most of the difference between a quiz and a game. Every arcade game you have ever played does this.

The bug to watch for

If the bar overshoots and jitters at the end, the easing step is too big. Say: **move it twelve percent of the remaining distance each frame**.

Task 4 ends when

You change a health number by hand in the code, reload, and watch the bar glide to the new value.

TASK 5

The question, and four answers

Now the content arrives, and where you put it matters.

PROMPT 4 · QUESTIONS

Put my questions in a **plain array at the very top of the file**, above all the game code, with a comment saying this is the part a teacher edits. Each one has a question, four answers, and which answer is right counting from zero. Below the canvas, show the current question and four answer buttons as **real HTML buttons, not drawn on the canvas**, so they work with a keyboard and a screen reader. Number them 1 to 4 and let those number keys answer. **Shuffle the answer order every time a question is asked**, and shuffle the question order every fight, so the same set is still usable on Friday.

Two decisions in that prompt worth understanding

Content at the top, above the code. A teacher editing next month should not have to go hunting past three hundred lines of game. This is the difference between a thing you use all year and a thing you built once.

Buttons in HTML, not painted on the canvas. Canvas text cannot be read by a screen reader or reached with Tab. Use the canvas for the fight and the page for the interface — that is not a compromise, it is the right call.

Shuffle, or it stops working on day two

Without it, the third student to play has memorised that the answer is always the second one. Shuffling answers *and* questions is what makes eight questions last a week.

Task 5 ends when

A question from your own unit is on screen with four buttons under it, in a different order each time you reload.

TASK 6

A right answer lands a hit

The first moment it is a game.

PROMPT 5 · DAMAGE

When the right answer is clicked, take health off the boss — about 14 to 18, with a little randomness so no two hits are identical. Then **disable all four buttons and highlight which one was right**, wait about a second, and ask the next question. Track a streak of correct answers in the state. **After three in a row, some hits are critical** and take roughly double, with a different message. Show the streak on screen once it is above one.

The critical hit is not decoration

It is the reason a class that is doing well starts paying *more* attention rather than less. A streak that pays off turns "we are winning" into "do not break the chain", and that is worth writing four lines for.

Randomness, but not too much

Identical damage every time feels mechanical. Wildly varying damage feels unfair. A small spread on a fixed base is the shape almost every game uses.

Task 6 ends when

You answer correctly and watch the boss bar drain.

TASK 7

A wrong answer hits back

Without stakes there is no game, only a quiz with a health bar on it.

PROMPT 6 · CONSEQUENCES

When a wrong answer is clicked, the class takes 12 to 17 damage and the streak resets to zero. Mark the clicked button wrong in red and **still highlight the right one in green**. Say something plain like "Not that one, look at the highlighted answer". **Wait longer before the next question after a miss than after a hit** — about 1.7 seconds instead of 0.9 — so there is time to actually read it.

The longer pause is the teaching

On a hit the class already knows; move on. On a miss the correct answer is on screen and every eye in the room is on it. **That extra eight-tenths of a second is the only reteaching this game does, and it is why it is a review game and not a scoreboard.** Ask for it explicitly, because no model will think of it.

Get the numbers right or the game is not fun

Boss 200 and about 15 damage a hit means **roughly twelve correct answers to win**. Class 100 and about 14 a miss means **about seven misses to lose**. So a class at sixty percent loses and a class at seventy-five percent squeaks through. Losable but winnable is the whole design. If your class wins every time, raise the boss health.

Task 7 ends when

You can lose. Answer badly on purpose and watch your own bar empty.

Make it feel good

Tasks 1 to 7 made a game. This is the one that makes a class shout at the screen.

PROMPT 7 · JUICE

Add three things when a hit lands. **Screen shake**: offset the whole canvas by a small random amount for a few frames and let it settle. **A white flash**: draw whoever got hit in solid white for about eight frames. **A floating number**: show the damage above them, drifting up and fading out. Make a critical hit shake harder and show a bigger number in gold. Keep all three short — a few frames, not a second.

These have a name, and the name is useful

Game developers call it **juice** — feedback that adds nothing to the rules and everything to how it feels. Almost none of the fun in an arcade game is in the rules. It is in the shake, the flash, the little number, the sound. **Say the word "juice" to an AI and it knows exactly what you want.**

The trap

Left to itself a model will make the shake last half a second and the flash a quarter of one. It looks broken. **Short and sharp.** If it feels wrong, ask for half as long before you ask for anything else.

Test it on somebody

Hand your laptop to a colleague without explaining anything and watch them answer three questions. If nobody reacts to a critical hit, the juice is too quiet.

Task 8 ends when

A critical hit makes you grin at your own screen, alone, at night.

TASK 9

Sound, with no sound files

The browser can make noise on its own. Nothing to download and nothing to lose.

PROMPT 8 · SOUND

Add sound using **the Web Audio API to generate tones — no audio files at all**. Write one small function that plays a tone at a given pitch for a given length. Then use it for: a hit, a critical hit as a quick rising run of notes, taking damage as a low buzz, winning as a short four-note fanfare, and losing as three falling notes. Keep everything quiet and under a quarter of a second except the endings. **Wrap it so the game keeps working if audio is blocked**.

Why no files

Audio files mean a folder, and a folder means the game is no longer one file you can email to the teacher next door. A tone generated in six lines of code travels with the game forever.

Browsers will not make noise until somebody clicks

This is a rule in every browser, not a bug in your game. Since your first sound happens after a button press, you will not notice — but if you ever add music to the title screen, that is why it is silent.

Test it on the classroom machine

School computers have their own opinions about volume. Check before the lesson, not during it.

Task 9 ends when

A hit makes a noise and a critical hit makes a better one.

TASK 10

Winning and losing

A game needs to end, and it needs to end differently.

PROMPT 9 · THE ENDINGS

When the boss reaches zero, show a win screen over the canvas: a headline, **how many were right out of how many, and how long it took**. When the class reaches zero, show a different screen that says so plainly and invites a rematch. Both have a button that starts a fresh fight with the questions reshuffled. Make Enter work on both.

Report the score, not just the result

"Eleven right out of thirteen, in four minutes" is a thing a class argues about and tries to beat. "You win" is a thing they watch. The number is what turns one fight into three.

Losing has to be survivable

The losing screen says what happened and offers another go. No scolding, no red X, no sad noise that goes on too long. A class that loses and immediately says "again" is the best outcome this game has.

Task 10 ends when

You have won once and lost once, and both times wanted to press the button.

TASK 11

The class record

One number, saved. It changes the whole week.

PROMPT 10 · THE RECORD

Save the fastest winning time to **localStorage** and show it on the title screen and the win screen. Only save it on a win. **Wrap every read and write in a try and catch so the game still works if storage is unavailable**, and show "No class record yet" when there is not one.

What this does to third period

First period sets a record. Second period sees it on the title screen. **By fifth period it is a competition between classes that you did not have to organize.** That is one afternoon of storage code doing more work than a display board.

Where the record lives, and what that means

In the browser, on that one computer. If you play on the classroom machine all day the record follows the day correctly. Playing on your laptop at home gives you a different record, and clearing browsing data resets it. For a bragging number that is exactly right — nothing important is lost when it goes.

Keep it to one number

It is tempting to store names against times. Do not. The moment a child's name is in there you have a student record on a shared classroom computer, and the bragging is not worth it. A time with no name is a leaderboard everybody can still argue about.

Task 11 ends when

You win, close the tab, reopen the file, and the record is still there.

TASK 12

Everyone can play it

The last task, and the one that decides whether it works in your actual room.

PROMPT 11 · REACH

Make sure this works for someone using **a keyboard instead of a mouse**: every answer reachable with Tab, usable with Enter, with a clearly visible outline on the selected one, and the first answer focused when a question appears. **Do not use color alone** to show right and wrong — the highlighted correct answer must be obvious without it. Make the buttons big enough for a finger, stack them in one column on a narrow screen, and check it fits a Chromebook screen with nothing cut off.

The Chromebook test is the real one

Teachers build on a laptop and deploy on a cart of Chromebooks with smaller screens. Open it on the actual hardware before the actual lesson. Something will be cut off and it takes two minutes to fix in advance and ruins a period to find out live.

Color alone is the most common accessibility bug in classroom games

Red for wrong and green for right is invisible to a good number of students. The fix costs nothing: the correct answer also gets a word, a mark, or a border change.

Task 12 ends when

You have played a whole fight using only the number keys, on the machine your class will actually use.

The six failures you will actually hit

In roughly the order they happen.

What you see	What it is, and the sentence that fixes it
A blank white page	One JavaScript error stops everything. Press F12, read the red line, paste it in with this is the error and this is what I was doing .
Blurry, soft-edged sprites	Smoothing is on. Say: set imageSmoothingEnabled to false and add image-rendering pixelated to the canvas in CSS .
The health bar jumps instead of draining	It is drawing the real value. Say: keep a separate shown value per bar and ease it toward the true one each frame .
Clicking fast answers twice	No lock. Say: ignore clicks until the next question is asked .
It wants me to run npm install	You got a framework. Say: rewrite this as one HTML file with no frameworks, no libraries and no build step .
It works, then the third change breaks it	The file has outgrown the chat. Start a fresh chat, paste the whole current file, then ask for the one change.

The habit worth more than the table: paste the error *with what you were doing when it happened*. "It broke" gets you guesses. "I clicked the second answer and the screen went white, here is the console" gets you the fix.

Every prompt, in one place – 1 to 6

Copy and paste. Square brackets mean put your own words in.

1 · THE STAGE

Build a single HTML file, no frameworks and no build step, that runs by opening it in a browser. Put a canvas in it, 640 by 360, scaled to fit, with `imageSmoothingEnabled` set to false. Set up a state object, a draw function, and a game loop using `requestAnimationFrame`, kept separate. Draw a dark sky, a horizon band and a floor, with stars at fixed positions. Make something move every frame so I can see the loop is alive.

2 · SPRITES

I want pixel sprites written as arrays of text, one character per pixel, a dot for transparent, so I can edit the art by typing. Write a function that takes one of those arrays, a color map, an x and y and a pixel size, and draws it. Make a small hero bottom left and a big boss at least fourteen pixels wide on the right, four or five times the hero. Give the boss a slow bob and a soft glow behind it.

3 · HEALTH BARS

Add health to the state: boss 200, class 100. Draw a bar for each with a label and numbers, boss at the top under its name, class at the bottom under the hero. The bars must slide toward the real value rather than jumping: keep a separate shown value per bar and move it twelve percent of the remaining distance each frame. Yellow below half, red below a quarter.

4 · QUESTIONS

Put my questions in a plain array at the very top of the file, above all the game code, with a comment saying this is the part a teacher edits. Each has a question, four answers, and which is right counting from zero. Below the canvas show the question and four real HTML buttons, not drawn on the canvas, numbered 1 to 4 with those number keys working. Shuffle the answers every question and the questions every fight.

5 · DAMAGE

When the right answer is clicked, take 14 to 18 off the boss with a little randomness. Disable all four buttons, highlight which was right, wait about a second, ask the next. Track a streak; after three in a row some hits are critical for roughly double with a different message. Show the streak once it is above one.

6 · CONSEQUENCES

When a wrong answer is clicked, the class takes 12 to 17 and the streak resets. Mark the clicked one red and still highlight the right one green. Say "Not that one, look at the highlighted answer". Wait about 1.7 seconds before the next question instead of 0.9, so there is time to read it.

PROMPTS

Every prompt, in one place – 7 to 12

7 · JUICE

Add three things when a hit lands. Screen shake: offset the whole canvas by a small random amount for a few frames and let it settle. A white flash: draw whoever got hit in solid white for about eight frames. A floating damage number that drifts up and fades. Criticals shake harder and show a bigger number in gold. Keep all three to a few frames, not a second.

8 · SOUND

Add sound using the Web Audio API to generate tones, no audio files at all. One small function that plays a tone at a given pitch for a given length. Use it for a hit, a critical as a quick rising run, taking damage as a low buzz, winning as a four-note fanfare, losing as three falling notes. Quiet, under a quarter second except the endings. Keep working if audio is blocked.

9 · THE ENDINGS

When the boss reaches zero show a win screen over the canvas with a headline, how many were right out of how many, and how long it took. When the class reaches zero show a different screen that says so plainly and invites a rematch. Both have a button that starts a fresh fight with questions reshuffled. Enter works on both.

10 · THE RECORD

Save the fastest winning time to localStorage and show it on the title and win screens. Only save on a win. Wrap every read and write in try and catch so the game works if storage is unavailable, and show "No class record yet" when there is not one.

11 · REACH

Make sure this works for someone using a keyboard instead of a mouse: every answer reachable with Tab, usable with Enter, a clearly visible outline on the selected one, first answer focused when a question appears. Do not use color alone to show right and wrong. Buttons big enough for a finger, one column on a narrow screen, and it must fit a Chromebook with nothing cut off.

12 · WHEN THREE ROUNDS HAVE NOT FIXED IT

[Start a fresh chat.] Here is my whole file. [paste it] Change only this one thing: [the thing]. Do not change anything else, and give me the whole file back.

What is in this download

boss-battle.html

The finished game, built by exactly these twelve tasks and played in a browser before publication. **Open it and play it first.** Knowing what the target feels like makes every prompt land better. It works as-is: change the eight questions at the top and it is your unit tonight.

Two ways to use this pack, both fine

Build it — work the twelve tasks and end up with something you understand well enough to change in March. **Play it** — swap the questions at the top and use it tomorrow, then read the tasks when you want to change something. The first teaches more. The second is a review game before homeroom.

The eight lines you edit

```
const BOSS_NAME = 'The Water Cycle';
const UNIT_NAME = 'Water Cycle Review';

const QUESTIONS = [
  { q: 'Your question here?',
    a: ['Right one', 'Wrong', 'Wrong', 'Wrong'], ok: 0 },
];
```

Everything else in the file is the game. You never have to touch it.

More questions is better. Eight works. Sixteen means a fight rarely repeats itself and the same file survives a whole unit.

THE QUESTIONS

How to write ones worth fighting over

This page matters more than any of the code. A game with lazy questions is a lazy game, however good the explosions are.

The rule: wrong answers should be the mistakes your students actually make

Weak: "Water turning to vapor is called?" — Evaporation / A dog / Tuesday / Purple.

Strong: Evaporation / Condensation / Precipitation / Transpiration. Every one is a real term from the unit, so getting it right means knowing the difference rather than spotting the joke.

Where to find those wrong answers

Your last quiz. The four things students actually put are better distractors than anything you or an AI will invent, **and you already have them**. Grade a stack, write down the four most common wrong answers, and the game writes itself.

Ask an AI for help the right way

Bad: "write me eight questions about the water cycle."

Better: "**Here are four questions and the wrong answers my students actually gave. Write four more in the same shape, where every wrong answer is a real misconception rather than an obvious joke.**" Do not send student work or names — type the misconceptions in yourself.

A mix that works

Two easy ones to get a streak going, four that are the actual content, two that are genuinely hard. Front-load an easy one so the first hit lands early and the room is in it.

Five ways to run it

1. Whole class, one screen

Project it. Cold-call who answers, or take a vote of hands first. **This is the one to start with** and the best use of the shared health bar — one class, one fate.

2. Two teams, alternating

Left side, right side, take turns. The class bar becomes whichever team is up. Loud, and worth it.

3. Individually on Chromebooks

Send the file or drop it in a shared folder — they open it and play alone at their own pace. No logins, nothing to join, works if the wifi is out.

4. Early finishers

Leave it open on a spare machine. Beat the class record.

5. A substitute can run it

"Open this file, project it, let them play." Nothing to log into and nothing to break. Worth leaving in the sub folder.

The thing to say on the first day: "I built this." Then take two minutes at the end to show them the eight lines with the questions in. Some of them will go home and try it, which is a better outcome than the review.

Building in somebody else's genre without stealing

The first thing most teachers want to make is a version of a game they love. Here is where the line actually sits.

The distinction that matters

Systems are not protected. Expression is. Turn-based fights, health bars, experience points, type advantages, critical hits, a four-option menu — those are rules, and rules are used by hundreds of games. **Names, characters, artwork, music, logos and written lines are protected**, and those are the things you must not take.

So you may freely

Build a turn-based fight with a menu of four moves · use health bars that drain and a critical hit · make a type chart of your own design · use badges that unlock areas · use the chunky four-shade handheld look, which is a hardware idiom hundreds of independent games use · write chiptune-style sounds of your own.

And you must never

Use a protected game's names, characters, creatures, places, items or move names — anywhere, including in your code and your file names · copy, trace or recolor its artwork or fonts · use its melodies, even altered · reuse its lines of dialogue, including the famous short ones. **Write your own in your own voice.**

The checklist, at the end of every session

Search your whole file for the protected names — there should be none · read every line the player sees out loud, and rewrite any that sounds like a line you have heard · look at each sprite and ask whether a fan would name it; if so, redraw it · hum your victory sound, and if it resolves into somebody else's tune, rewrite it.

And the practical point. A classroom game you never publish is a very different risk from one you sell or put online. Keep it in your room and this is mostly about teaching students to do it properly. Put it on the internet and the checklist is not optional. **This is not legal advice — it is the line a careful teacher can hold.**

The shortest privacy page in this series

Because the honest answer is: there is almost nothing to say, and that is the point.

What this game knows about your students

Nothing. No names, no accounts, no scores against people, no roster. It cannot know anything, because there is nowhere to type it and nothing to type it into.

What leaves the room

Nothing. The file is on the computer it is running on. There is no server, no network call, no analytics and no company behind it. Unplug the internet and it plays exactly the same.

The one thing it saves

A single number — the fastest winning time — in that browser on that machine. No name attached, by design. If somebody clears browsing data it is gone, and nothing is lost that matters.

Keep it that way

The obvious next feature is storing who got the high score. Do not. That single change turns a toy with no data into a student record on a shared classroom computer, and the bragging is not worth the conversation. **A time with nobody's name on it is still something a class will fight over.**

Worth saying to an administrator in one sentence: "It is a file on my computer that has no student information in it and does not connect to anything." That is the whole disclosure, and it is true.

Eight things to try before a class sees it

Ten minutes at your desk saves a period.

Try this	You want
Answer everything right	A win in about twelve questions
Answer everything wrong	A loss in about seven, and a rematch offered
Get three right in a row	A critical hit, louder and bigger
Click an answer twice, fast	The second click ignored
Play using only the number keys	A whole fight, no mouse
Win, close the tab, reopen it	The class record still there
Start four fights in a row	A different first question most times
Open it on the classroom machine	Nothing cut off, sound audible

The one everybody skips

The Chromebook. You built it on a laptop. Your class does not have your laptop. Two minutes now, or a ruined period later.

And one worth doing with a person

Hand it to a colleague and say nothing at all. If they hesitate about what to do, your first question or your buttons need work — not their attention.

Six changes that take one prompt each

The game you built is a starting point. Each of these is one sentence to an AI.

Re-skin it

"Change the palette to these colors and redraw the boss as [a thing from my unit]." The sprites are text arrays, so a new boss is a few lines.

More than one boss

"After the boss dies, a tougher second one appears with more health." A unit becomes a gauntlet.

Let them heal

"Three right in a row heals the class ten." Rewards a run and keeps a struggling class alive longer.

A timer per question

"Ten seconds to answer, and running out counts as wrong." Use carefully — it raises the stakes and it raises the anxiety.

Categories as moves

"Four buttons for four topics; picking one draws a question from that topic." Now the class is choosing strategy, not just answering.

Two-player

"Two teams alternate turns and each has its own health bar." The loudest version of this game.

Change one thing at a time and play it after each one. Two changes in one prompt means two things to untangle when it breaks, and it will break.

How this pack is built, and what it does not claim

Where this comes from

Not theory. I build software for my own high school shop course and run it with real students all year, including a full retro game with its own design document and a written set of rules about what may and may not be borrowed from the genre it sits in. The IP page in this pack is that thinking, reduced to the part a teacher needs.

What was tested before this went out

The included game was played in a real browser, start to finish, both ways. A perfect run won in **10 answers** and the class bar never moved. A run of deliberate wrong answers lost in **8**. The number keys answered, the class record survived a reload, and four fights in a row opened with four different questions. No console errors anywhere.

What is NOT claimed

- **Not a replacement for Blooket or Gimkit.** They are faster to start and more polished. The trade is on page 3, stated in both directions.
- **Not a gradebook.** It records no scores against any student and should not.
- **Not legal advice** about copyright. The IP page is how a careful teacher can hold the line, not a substitute for a lawyer if you intend to publish.
- **Not guaranteed to come out identical.** You are working with a model; your boss will look different from mine. That is the point.

Why the prompts spend so much time saying no

No frameworks. No build step. No audio files. No color-only signals. Left alone a model builds what most software looks like, and most software has more parts than a teacher can maintain. **Saying no is the part of prompting nobody teaches, and it is most of the skill.**

ONE LAST THING

If this was useful

Leaving a review earns you TPT credit

TPT gives you credit toward future purchases for every review you leave, and reviews are how other teachers find work like this. Thirty seconds.

If it did not work, tell me before you rate it

The Q&A on the listing reaches me directly. A game that will not open is almost always one line, and I would far rather fix it than have you stuck with a file that does nothing.

The one thing to take away

The fun is not in the rules. It is in the shake, the flash, the number and the sound. That is true of every game you have played, it takes about twenty lines, and it is the difference between something your class tolerates and something they ask for.

Built and audited by two certified teachers.

Every prompt here was run. The game was played through to a win and to a loss before this went out, on the file that is in this download.